

C# 프로그래밍

동명대학교 게임학부 · 2026 년 1 학기

Lecture 6

절차적 vs OOP

클래스가 코딩을 간편하게 하는 이유

강영민

동명대학교 게임학부

실습 환경: Visual Studio Code + .NET SDK

Contents

1 강의 개요	2
1.1 학습 목표	2
1.2 강의 구성	2
2 A부: 숫자 야구 — 절차적 vs OOP	3
2.1 Step A-Procedural: 절차적으로 완성이기	3
2.2 Step A-OOP-1: SecretNum 클래스 — 데이터 분리	5
2.3 Step A-OOP-2: Checker 클래스 — 입력과 판정 분리	6
2.4 Step A-OOP-3: GameManager로 게임 완성	7
3 B부: 행맨 — 절차적 vs OOP	10
3.1 Step B-Procedural: 절차적으로 완성이기	10
3.2 Step B-OOP-1: WordClass — 단어 관리 분리	13
3.3 Step B-OOP-2: HangmanDrawing — 그림 출력 분리	14
3.4 Step B-OOP-3: CheckWord — 빈칸 관리 분리	14
3.5 Step B-OOP-4: GameManager로 행맨 완성	15
4 종합 비교: 왜 클래스를 쓰는가?	18
4.1 코드 줄 수 비교	18
4.2 수정 시뮬레이션	18
4.3 클래스를 쓰면 좋은 이유 3가지	19
5 핵심 개념 정리	20
5.1 절차적 프로그래밍 vs 객체지향 프로그래밍	20
5.2 이번 강의에서 확인한 OOP 개념	20
5.3 두 게임의 클래스 구조 비교	20

1. 강의 개요

1.1. 학습 목표

이번 강의에서는 Lecture 5에서 만들었던 **숫자야구**와 **행맨** 게임을 두 가지 방식으로 다시 만들어 보면서, 클래스가 코딩을 얼마나 간편하게 해 주는지를 체험한다.

- 클래스 없이(절차적으로) 게임을 완성해 보고, 코드의 복잡함을 직접 경험한다.
- 같은 게임을 클래스로 단계별 분리하며, 코드가 어떻게 깔끔해지는지 비교한다.
- “왜 클래스를 쓰는가?”라는 질문에 구체적 근거를 들어 답할 수 있다.
- 수정·확장·재사용 관점에서 절차적 코드와 OOP 코드의 차이를 설명할 수 있다.

1.2. 강의 구성

파트	게임	내용
A부	숫자 야구	절차적 완성 → OOP로 3단계 리팩토링
B부	행맨	절차적 완성 → OOP로 4단계 리팩토링
종합	—	줄 수 비교, 수정 시뮬레이션, 핵심 정리

이번 강의의 핵심 질문

“같은 게임인데, 왜 클래스를 쓰면 코딩이 편해질까?”

이 질문에 답하기 위해, 같은 기능을 가진 게임을 두 가지 방식으로 만들고 직접 비교한다.

- **절차적(Procedural)**: 클래스 없이, 변수와 배열만으로 처음부터 끝까지 한 덩어리로 작성
- **OOP(Object-Oriented)**: 역할별로 클래스를 나누어 단계적으로 조합

실행 결과는 완전히 동일하다. 달라지는 것은 코드의 구조뿐이다!

2. A부: 숫자 야구 — 절차적 vs OOP

숫자 야구는 비밀 세 자리 숫자(1~9, 중복 없음)를 맞추는 게임이다. 같은 자리·같은 숫자면 스트라이크(S), 다른 자리·같은 숫자면 볼(B)이다.

먼저 클래스 없이 게임을 완성한 뒤, 같은 게임을 클래스로 분리해 본다.

2.1. Step A-Procedural: 절차적으로 완성하기

클래스를 사용하지 않고, 변수·배열·반복문만으로 게임을 완성한다. 코드를 읽으면서 “이 부분이 무슨 역할인지” 구분해 보자.

```

1  // --- 1. Generate secret number (3 unique digits, 1~9) ---
2  Random rand = new Random();
3  List<int> pool = new List<int>
4      { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
5  int secretA = pool[rand.Next(pool.Count)];
6  pool.Remove(secretA);
7  int secretB = pool[rand.Next(pool.Count)];
8  pool.Remove(secretB);
9  int secretC = pool[rand.Next(pool.Count)];
10 int[] secret = { secretA, secretB, secretC };
11
12 Console.WriteLine("=== Number Baseball (Procedural) ===");
13 Console.WriteLine("Guess 3 unique digits from 1~9!\n");
14
15 int turn = 0;
16
17 // --- 2. Game loop ---
18 while (true)
19 {
20     // 2-1. Read user input
21     Console.Write("Enter 3 digits (e.g. 1 2 3): ");
22     string line = Console.ReadLine();
23     string[] parts = line.Split(' ');
24
25     // 2-2. Validate: must be 3 parts
26     if (parts.Length != 3)
27     {
28         Console.WriteLine(
29             " [Error] Enter 3 numbers separated by spaces!");
30         continue;
31     }
32
33     // 2-3. Validate: parse + range check
34     int[] guess = new int[3];
35     bool inputOk = true;
36     for (int i = 0; i < 3; i++)
37     {
38         if (!int.TryParse(parts[i], out guess[i]))
39         {
40             Console.WriteLine(" [Error] Enter numbers!");
41             inputOk = false;

```

```

42         break;
43     }
44     if (guess[i] < 1 || guess[i] > 9)
45     {
46         Console.WriteLine(
47             " [Error] Only 1~9 allowed!");
48         inputOk = false;
49         break;
50     }
51 }
52 if (!inputOk) continue;
53
54 turn++;
55
56 // 2-4. Calculate strike / ball
57 int strike = 0;
58 int ball = 0;
59 for (int i = 0; i < 3; i++)
60 {
61     for (int j = 0; j < 3; j++)
62     {
63         if (guess[i] == secret[j])
64         {
65             if (i == j) strike++;
66             else ball++;
67         }
68     }
69 }
70
71 // 2-5. Print result
72 if (strike == 3)
73 {
74     Console.WriteLine(" * 3S! Correct!");
75     Console.WriteLine(" * " + turn + " tries!");
76     break;
77 }
78 else if (strike == 0 && ball == 0)
79     Console.WriteLine(" -> Out!");
80 else
81     Console.WriteLine(" -> " + strike
82         + "S " + ball + "B");
83 }

```

절차적 코드의 특징

- 코드 전체가 약 75줄의 한 덩어리로 이루어져 있다.
- 비밀 숫자 생성, 입력 검증, S/B 판정, 결과 출력이 모두 섞여 있다.
- 코드를 처음 보는 사람은 “어디부터 어디까지가 입력 처리인지” 찾기 어렵다.

- “4자리로 바꿔주세요” 같은 수정 요청이 오면 여러 곳을 동시에 수정해야 한다.

2.2. Step A-OOP-1: SecretNum 클래스 — 데이터 분리

절차적 코드에서 “비밀 숫자” 관련 부분을 클래스로 분리한다.

분리 전 vs 분리 후

절차적 int[] secret에 값을 넣고, 검증은 별도로 직접 작성
 OOP SecretNum이 저장 + 검증을 한 곳에서 관리
 → 잘못된 값이 저장될 수 없다 (검증이 내장되어 있으므로)

```

1 class SecretNum
2 {
3     private int[] digits;
4
5     public SecretNum()
6     {
7         digits = new int[3];
8     }
9
10    // Store 3 digits with validation
11    public bool SetNumber(int a, int b, int c)
12    {
13        if (a < 1 || a > 9 || b < 1 || b > 9
14            || c < 1 || c > 9)
15        {
16            Console.WriteLine(
17                " [Error] Only 1~9 allowed!");
18            return false;
19        }
20        if (a == b || b == c || a == c)
21        {
22            Console.WriteLine(
23                " [Error] Duplicate number!");
24            return false;
25        }
26        digits[0] = a; digits[1] = b; digits[2] = c;
27        return true;
28    }
29
30    public int GetDigit(int index)
31    {
32        return digits[index];
33    }
34
35    public string GetNumber()
36    {
37        return digits[0] + " " + digits[1]
38            + " " + digits[2];
  
```

```

39     }
40 }

```

클래스의 첫 번째 장점: 안전한 데이터

절차적 코드에서는 `secret[0] = -5`; 같은 잘못된 값을 아무런 검증 없이 넣을 수 있다. `SecretNum`은 `SetNumber()`를 통해서만 값을 설정할 수 있으며, 1~9 범위와 중복 여부를 자동으로 검증한다. 이것이 캡슐화(Encapsulation)의 효과이다.

2.3. Step A-OOP-2: Checker 클래스 — 입력과 판정 분리

이번에는 “사용자 입력”과 “S/B 판정” 부분을 Checker 클래스로 분리한다.

절차적 코드에서 빠지는 부분

절차적 코드의 2-1~2-4 부분 (약 35줄)이 Checker 안으로 이동한다. 메인 코드에서는 다음 2줄만 남는다:

```

int[] input = checker.GetInputNumbers();
int[] result = checker.Check(input[0], input[1], input[2]);

```

Checker 클래스의 핵심 메서드

```

1  class Checker
2  {
3      private SecretNum target;
4
5      public Checker(SecretNum target)
6      {
7          this.target = target;
8      }
9
10     // Get 3 numbers from user (null if invalid)
11     public int[] GetInputNumbers()
12     {
13         Console.Write("Enter 3 digits (e.g. 1 2 3): ");
14         string line = Console.ReadLine();
15         string[] parts = line.Split(' ');
16
17         if (parts.Length != 3)
18         {
19             Console.WriteLine(" [Error] Need 3 numbers!");
20             return null;
21         }
22
23         int[] nums = new int[3];
24         for (int i = 0; i < 3; i++)
25         {
26             if (!int.TryParse(parts[i], out nums[i]))
27             {

```

```

28         Console.WriteLine(
29             " [Error] Enter numbers!");
30         return null;
31     }
32     if (nums[i] < 1 || nums[i] > 9)
33     {
34         Console.WriteLine(
35             " [Error] Only 1~9 allowed!");
36         return null;
37     }
38 }
39 return nums;
40 }
41
42 // Strike/Ball judgment
43 public int[] Check(int a, int b, int c)
44 {
45     int[] guess = { a, b, c };
46     int strike = 0;
47     int ball = 0;
48
49     for (int i = 0; i < 3; i++)
50     {
51         for (int j = 0; j < 3; j++)
52         {
53             if (guess[i] == target.GetDigit(j))
54             {
55                 if (i == j) strike++;
56                 else ball++;
57             }
58         }
59     }
60     return new int[] { strike, ball };
61 }
62 }

```

클래스의 두 번째 장점: 복잡한 로직 숨기기

입력 파싱(Split, TryParse, 범위 검증)과 판정(이중 for문) 로직은 약 35줄이다. 이 35줄이 Checker 클래스 안에 들어가면, 게임 루프에서는 보이지 않는다. 게임 루프는 “입력 받기” → “판정하기”라는 흐름만 읽히게 된다.

2.4. Step A-OOP-3: GameManager로 게임 완성

마지막으로 GameManager가 SecretNum과 Checker를 조합하여 게임을 완성한다.

```

1 class GameManager
2 {
3     private SecretNum secret;
4     private Checker checker;
5 }

```



```

6     public GameManager()
7     {
8         secret = new SecretNum();
9         // Pick 3 unique numbers from 1~9
10        Random rand = new Random();
11        List<int> pool = new List<int>
12            { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
13        int a = pool[rand.Next(pool.Count)];
14        pool.Remove(a);
15        int b = pool[rand.Next(pool.Count)];
16        pool.Remove(b);
17        int c = pool[rand.Next(pool.Count)];
18        secret.SetNumber(a, b, c);
19        checker = new Checker(secret);
20    }
21
22    public void Play()
23    {
24        Console.WriteLine("=== Number Baseball (OOP) ===");
25        Console.WriteLine(
26            "Guess 3 unique digits from 1~9!\n");
27
28        int turn = 0;
29        while (true)
30        {
31            // 1. Input -- delegate to Checker
32            int[] input = checker.GetInputNumbers();
33            if (input == null) continue;
34            turn++;
35
36            // 2. Judgment -- delegate to Checker
37            int[] result = checker.Check(
38                input[0], input[1], input[2]);
39            int strike = result[0];
40            int ball = result[1];
41
42            // 3. Print result
43            if (strike == 3)
44            {
45                Console.WriteLine(" * 3S! Correct!");
46                Console.WriteLine(
47                    " * " + turn + " tries!");
48                break;
49            }
50            else if (strike == 0 && ball == 0)
51                Console.WriteLine(" -> Out!");
52            else
53                Console.WriteLine(" -> " + strike
54                    + "S " + ball + "B");
55        }
56    }

```

57 }

main 코드 — 단 2줄!

```

1 GameManager manager = new GameManager();
2 manager.Play();

```

숫자야구: 절차적 vs OOP 비교

비교 항목	절차적	OOP
전체 코드	약 75줄 한 덩어리	역할별 3개 클래스
게임 루프	약 55줄	Play() 약 30줄
입력 검증 로직	게임 루프 안에 섞임	Checker 안에 격리
판정 로직	게임 루프 안에 섞임	Checker.Check()에 격리
main 코드	전체가 main	2줄

실행 결과는 완전히 동일하지만, 코드의 읽기 쉬움이 전혀 다르다!

실습 A: 숫자 야구

1. StepA_Baseball_Procedural_01을 실행하고 게임을 해 보자.
2. StepA_Baseball_OOP_01, 02, 03을 순서대로 실행해 보자.
3. OOP-3의 실행 결과가 Procedural과 동일한지 확인하자.
4. 절차적 코드에서 “4자리 숫자 야구로 바꾸려면” 어디를 수정해야 하는지 찾아보자.
→ OOP 버전에서는 어떤 클래스만 수정하면 되는지 비교하자.

3. B부: 행맨 — 절차적 vs OOP

행맨은 비밀 단어의 알파벳을 한 글자씩 맞추는 게임이다. 숫자야구보다 요소가 많아서 (단어 검사, 그림 출력, 빈칸 관리), 절차적 코드와 OOP 코드의 차이가 더욱 극명하게 드러난다.

3.1. Step B-Procedural: 절차적으로 완성하기

클래스 없이 게임을 완성한다. 코드가 얼마나 길어지는지 주목하자.

1. 초기화: 단어 선택 + 빈칸 배열 + 그림 데이터

```

1 // --- 1. Word list + random selection ---
2 string[] wordList = { "APPLE", "BANANA", "CHERRY",
3   "DRAGON", "FOREST", "GUITAR", "HAMMER",
4   "ISLAND", "JUNGLE", "KNIGHT" };
5 Random rand = new Random();
6 string secretWord = wordList[rand.Next(wordList.Length)];
7
8 // --- 2. Create blank display array ---
9 char[] display = new char[secretWord.Length];
10 for (int i = 0; i < display.Length; i++)
11     display[i] = '_';
12
13 // --- 3. Hangman drawing data (by life) ---
14 string[][] stages = new string[][]
15 {
16     // life 0: dead
17     new[] { " +---+ ",
18             " |  O  ",
19             " | *|\\* Failed!",
20             " | / \\ ",
21             " +-----" },
22     // life 1
23     new[] { " +---+ ", " |  O  ",
24             " | *|\\*", " | /  ",
25             " +-----" },
26     // ... (life 2~7 omitted for brevity) ...
27     // life 8: start (empty gallows)
28     new[] { " +---+ ", " |      ",
29             " |      ", " |      ",
30             " +-----" },
31 };

```

여기서 벌써 문제가 보인다

그림 데이터(stages)만 약 50줄을 차지한다. 아직 게임 로직은 한 줄도 작성하지 않았는데, 코드가 이미 길다. 이 50줄이 게임 코드 중간에 끼어 있으면 “게임 흐름”을 읽기 어렵다.

2. 게임 루프

```
1 int life = 8;
2 int turn = 0;
3 bool complete = false;
4
5 while (life > 0 && !complete)
6 {
7     // 4-1. Draw hangman
8     foreach (string s in stages[life])
9         Console.WriteLine(s);
10
11     // 4-2. Show current display
12     string displayStr = "";
13     for (int i = 0; i < display.Length; i++)
14     {
15         if (i > 0) displayStr += " ";
16         displayStr += display[i];
17     }
18     Console.WriteLine("  Word: " + displayStr);
19     Console.WriteLine("  Lives: " + life);
20
21     // 4-3. Get user input
22     Console.Write("Enter a letter: ");
23     string input = Console.ReadLine();
24     if (input.Length == 0)
25     {
26         Console.WriteLine("  [Error] Enter a letter!");
27         continue;
28     }
29     char c = char.ToUpper(input[0]);
30     if (!char.IsLetter(c))
31     {
32         Console.WriteLine(
33             "  [Error] Enter an alphabet!");
34         continue;
35     }
36     turn++;
37
38     // 4-4. Check letter + fill blanks
39     bool hit = false;
40     for (int i = 0; i < secretWord.Length; i++)
41     {
42         if (secretWord[i] == c)
43         {
44             display[i] = c;
45             hit = true;
46         }
47     }
48
49     if (hit)
```

```

50     Console.WriteLine("    -> Correct!\n");
51     else
52     {
53         life--;
54         Console.WriteLine("    -> Wrong! (lives -1)\n");
55     }
56
57     // 4-5. Check completion
58     complete = true;
59     for (int i = 0; i < display.Length; i++)
60     {
61         if (display[i] == '_')
62         {
63             complete = false;
64             break;
65         }
66     }
67 }

```

3. 최종 결과 출력

```

1 // --- 5. Final result ---
2 foreach (string s in stages[life])
3     Console.WriteLine(s);
4
5 // Display string (same code AGAIN!)
6 string finalDisplay = "";
7 for (int i = 0; i < display.Length; i++)
8 {
9     if (i > 0) finalDisplay += " ";
10    finalDisplay += display[i];
11 }
12 Console.WriteLine("    Word: " + finalDisplay);
13
14 if (complete)
15 {
16     Console.WriteLine("    * Correct! Answer: "
17         + secretWord);
18     Console.WriteLine("    * " + turn + " tries!");
19 }
20 else
21     Console.WriteLine("    Game Over! Answer: "
22         + secretWord);

```

절차적 행맨의 문제점 총정리

1. 총 약 130줄 — 하나의 게임치고 매우 길다.
2. 그림 데이터 50줄이 게임 코드 한가운데에 있어서 흐름이 끊긴다.

3. 빈칸 표시 문자열 생성 코드가 2번 반복된다 (루프 안 + 최종 결과).
4. 단어 검사, 빈칸 채우기, 완성 확인, 그림 출력이 모두 섞여 있다.
5. “그림 스타일을 바꾸고 싶다” → 이 130줄 전체에서 해당 부분을 찾아야 한다.

3.2. Step B-OOP-1: WordClass — 단어 관리 분리

먼저 “단어 저장 + 알파벳 검색” 기능을 WordClass로 분리한다.

```

1  class WordClass
2  {
3      private string word;
4
5      public WordClass() { word = ""; }
6
7      public void SetString(string s)
8      {
9          word = s.ToUpper();
10     }
11     public string GetWord() { return word; }
12     public int GetLength() { return word.Length; }
13
14     // Return positions of char c as int array
15     public int[] CheckString(char c)
16     {
17         c = char.ToUpper(c);
18
19         // Step 1: count
20         int count = 0;
21         for (int i = 0; i < word.Length; i++)
22             if (word[i] == c) count++;
23
24         // Step 2: build position array
25         int[] positions = new int[count];
26         int idx = 0;
27         for (int i = 0; i < word.Length; i++)
28         {
29             if (word[i] == c)
30             {
31                 positions[idx] = i;
32                 idx++;
33             }
34         }
35         return positions;
36     }
37 }
```

절차적 코드와 비교

절차적	for (int i = 0; i < secretWord.Length; i++) if (secretWord[i] == c) ... — 매 번 같은 for문을 직접 작성
OOP	int[] pos = wc.CheckString('A'); — 한 줄!

3.3. Step B-OOP-2: HangmanDrawing — 그림 출력 분리

절차적 코드에서 약 50줄을 차지하던 그림 데이터를 HangmanDrawing 클래스로 격리한다.

```

1 class HangmanDrawing
2 {
3     // stages[0]=dead ~ stages[8]=start
4     private string[][] stages = new string[][] { ... };
5
6     public void Draw(int life)
7     {
8         if (life < 0) life = 0;
9         if (life > 8) life = 8;
10        foreach (string line in stages[life])
11            Console.WriteLine(line);
12    }
13 }
```

50줄이 1줄로!

절차적	string[][] stages = ... (약 50 줄) + foreach (string s in stages[life]) Console.WriteLine(s);
OOP	drawing.Draw(life); — 한 줄!

50줄의 그림 데이터가 HangmanDrawing 안에 숨겨지면, 게임 코드에서는 “그림을 그려라”라는 의도만 읽히게 된다.

3.4. Step B-OOP-3: CheckWord — 빈칸 관리 분리

“빈칸 배열 만들기, 글자 채우기, 표시 문자열 생성, 완성 확인”을 CheckWord 클래스로 분리한다.

```

1 class CheckWord
2 {
3     private WordClass target;
4     private char[] display;    // '_' or letter
5
6     public CheckWord(WordClass target)
7     {
8         this.target = target;
9         display = new char[target.GetLength()];
```

```

10     for (int i = 0; i < display.Length; i++)
11         display[i] = '_';
12     }
13
14     // Get one letter from user (' ' if invalid)
15     public char GetInputChar() { ... }
16
17     // Fill matched positions, return true if hit
18     public bool Check(char c)
19     {
20         int[] positions = target.CheckString(c);
21         if (positions.Length == 0) return false;
22
23         for (int i = 0; i < positions.Length; i++)
24             display[positions[i]] = c;
25         return true;
26     }
27
28     // "B A _ A _ A" format
29     public string GetDisplay() { ... }
30
31     // All blanks filled?
32     public bool IsComplete() { ... }
33 }

```

반복 코드가 사라진다

절차적 코드에서 빈칸 표시 문자열 생성 코드가 2번 반복되었던 것을 기억하는가? CheckWord.GetDisplay() 하나로 통일되어, 어디서든 한 줄로 호출하면 된다.

checker.Check(c)	글자 채우기
checker.GetDisplay()	현재 상태 문자열
checker.IsComplete()	완성 여부 확인

절차적 코드에서 4~5개의 for문으로 흩어져 있던 로직이 각각 한 줄 호출로 대체된다!

3.5. Step B-OOP-4: GameManager로 행맨 완성

GameManager가 세 클래스를 조합하여 게임을 완성한다.

```

1  class GameManager
2  {
3      private WordClass word;
4      private HangmanDrawing drawing;
5      private CheckWord checker;
6      private int life;
7
8      private static string[] wordList =
9          { "APPLE", "BANANA", "CHERRY", "DRAGON",
10            "FOREST", "GUITAR", "HAMMER", "ISLAND",

```



```

11         "JUNGLE", "KNIGHT" };
12
13     public GameManager()
14     {
15         word = new WordClass();
16         string picked =
17             wordList[new Random().Next(wordList.Length)];
18         word.SetString(picked);
19         drawing = new HangmanDrawing();
20         checker = new CheckWord(word);
21         life = 8;
22     }
23
24     public void Play()
25     {
26         Console.WriteLine("=== Hangman (OOP) ===");
27         Console.WriteLine("Guess the word letter by letter!");
28         Console.WriteLine("Lives: " + life + "\n");
29
30         int turn = 0;
31         while (life > 0 && !checker.IsComplete())
32         {
33             // Draw -- delegate to HangmanDrawing
34             drawing.Draw(life);
35             Console.WriteLine();
36             Console.WriteLine("  Word: "
37                 + checker.GetDisplay());
38             Console.WriteLine("  Lives: " + life);
39
40             // Input -- delegate to CheckWord
41             Console.WriteLine();
42             char input = checker.GetInputChar();
43             if (input == ' ') continue;
44             turn++;
45
46             // Judgment -- delegate to CheckWord
47             bool hit = checker.Check(input);
48             if (hit)
49                 Console.WriteLine("  -> Correct!\n");
50             else
51             {
52                 life--;
53                 Console.WriteLine(
54                     "  -> Wrong! (lives -1)\n");
55             }
56         }
57
58         // Final result
59         drawing.Draw(life);
60         Console.WriteLine("\n  Word: "
61             + checker.GetDisplay() + "\n");

```

```

62
63     if (checker.IsComplete())
64     {
65         Console.WriteLine(" * Correct! Answer: "
66             + word.GetWord());
67         Console.WriteLine(
68             " * " + turn + " tries!");
69     }
70     else
71         Console.WriteLine(" Game Over! Answer: "
72             + word.GetWord());
73     }
74 }

```

main 코드 — 동일한 패턴 2줄

```

1 GameManager manager = new GameManager();
2 manager.Play();

```

행맨: 절차적 vs OOP 비교

비교 항목	절차적	OOP
전체 코드	약 130줄 한 덩어리	역할별 4개 클래스
게임 루프	약 60줄	Play() 약 35줄
그림 데이터	게임 코드 한가운데 (50줄)	HangmanDrawing 안에 격리
빈칸 표시 코드	2번 반복	GetDisplay() 한 줄
단어 검사 코드	for문 직접 작성	CheckString() 한 줄
main 코드	전체가 main	2줄

실습 B: 행맨

1. StepB_Hangman_Procedural_01을 실행하고 게임을 해 보자.
2. StepB_Hangman_OOP_01~04를 순서대로 실행해 보자.
3. 절차적 코드에서 “빈칸 표시 문자열 생성”이 2번 반복되는 곳을 찾아보자.
→ OOP에서는 어떻게 해결되었는지 확인하자.
4. OOP-4에서 단어 목록에 5개를 추가해 보자. 수정할 곳이 몇 군데인가?

4. 종합 비교: 왜 클래스를 쓰는가?

두 게임을 절차적/OOP 두 가지 방식으로 만들어 보았다. 이제 구체적인 숫자와 시나리오로 차이를 정리한다.

4.1. 코드 줄 수 비교

	절차적	OOP (게임 루프)	차이
숫자 야구	약 75줄	Play() 약 30줄	-45줄
행맨	약 130줄	Play() 약 35줄	-95줄

줄 수가 줄어드는 게 아니다!

클래스 전체를 합하면 OOP가 오히려 줄 수가 더 많을 수 있다. 줄어드는 것은 한 번에 읽어야 하는 양이다.

- 절차적: 75줄(야구) / 130줄(행맨)을 한꺼번에 이해해야 한다.
- OOP: Play()의 30~35줄만 읽으면 게임 흐름을 파악할 수 있다. 상세 로직은 필요할 때 해당 클래스만 열어보면 된다.

4.2. 수정 시뮬레이션

실제로 수정 요청이 왔을 때, 어느 쪽이 더 편한지 비교해 보자.

시나리오 1: “숫자야구를 4자리로 바꿔주세요”

	절차적	OOP
수정 범위	비밀 숫자 생성 부분, 입력 검증 부분, 판정 부분 — 코드 곳곳을 찾아 수정	SecretNum과 Checker만 수정. GameManager는 변경 없음
실수 위험	한 곳이라도 빼먹으면 버그	해당 클래스만 보면 됨

시나리오 2: “행맨 그림을 다른 스타일로 바꿔주세요”

	절차적	OOP
수정 범위	130줄 코드 중간에 있는 stages 배열을 찾아서 수정	HangmanDrawing 클래스 하나만 수정
다른 코드 영향	stages 주변 코드를 건드릴 위험	다른 클래스에 영향 없음

시나리오 3: “행맨 그림을 다른 게임에서도 쓰고 싶어요”

	절차적	OOP
방법	stages 배열과 출력 코드를 복사-붙여넣기	HangmanDrawing 클래스를 그대로 가져다 쓰기
문제	복사본이 여러 개 생김 → 수정 시 전부 고쳐야 함	원본 하나만 수정하면 모든 곳에 반영

4.3. 클래스를 쓰면 좋은 이유 3가지

정리: 클래스의 세 가지 장점

1. 읽기 쉽다 (가독성)

- 게임 루프가 “입력 → 판정 → 출력”이라는 흐름만 보여준다.
- 상세 로직은 클래스 안에 숨겨져 있어, 필요할 때만 열어본다.

2. 고치기 쉽다 (유지보수성)

- 수정 범위가 해당 클래스 하나로 한정된다.
- 다른 클래스에 영향을 주지 않으므로 실수 위험이 적다.

3. 재사용 가능하다 (재사용성)

- HangmanDrawing을 다른 프로젝트에 그대로 가져갈 수 있다.
- SecretNum을 4자리 숫자야구에도 활용할 수 있다.

게임이 복잡할수록 차이가 커진다

숫자야구에서는 절차적 75줄 vs OOP 30줄로, 차이가 있긴 하지만 “그럭저럭 읽을 만”했다. 행맨에서는 절차적 130줄 vs OOP 35줄로, 차이가 거의 4배가 되었다. 실제 게임(예: Unity 프로젝트)은 수천~수만 줄이다. 그 규모에서 클래스 없이 코딩하면 어떤 일이 벌어질지 상상해 보자!

5. 핵심 개념 정리

5.1. 절차적 프로그래밍 vs 객체지향 프로그래밍

	절차적 (Procedural)	객체지향 (OOP)
코드 구조	위에서 아래로 한 덩어리	역할별 클래스로 분리
데이터 관리	변수/배열을 직접 관리	클래스 안에 캡슐화
코드 흐름	전체를 읽어야 파악	메인 흐름만 읽으면 파악
수정 범위	코드 전체를 살펴야 함	해당 클래스만 수정
재사용	복사-붙여넣기	클래스를 그대로 가져다 씀
규모 적합성	소규모 스크립트에 적합	중~대규모 프로젝트에 적합

5.2. 이번 강의에서 확인한 OOP 개념

개념	이번 강의에서의 적용
캡슐화	SecretNum의 private 배열 — 잘못된 값이 들어갈 수 없음
클래스 간 협력	Checker가 SecretNum을, CheckWord가 WordClass를 필드로 보관
역할 분리	데이터 / 판정 / 출력 / 게임 관리를 각각 다른 클래스가 담당
GameManager 패턴	여러 클래스를 조합하여 게임 루프를 관리 — main은 2줄

5.3. 두 게임의 클래스 구조 비교

숫자야구 vs 행맨: 같은 패턴, 다른 클래스

[A: Number Baseball]	[B: Hangman]
SecretNum (data)	WordClass (data)
Checker (input + judge)	CheckWord (input + blank mgmt)
GameManager (game loop)	HangmanDrawing (display)
	GameManager (game loop)

두 게임 모두 “데이터 → 판정 → GameManager” 패턴을 따른다. 행맨은 출력(HangmanDrawing)이 추가로 분리되었을 뿐, 기본 구조는 동일하다.

도전 과제

1. 절차적 코드 수정 체험: 절차적 숫자야구를 4자리로 바꿔보라. 몇 곳을 수정해야 했는가? OOP 버전에서는 몇 곳인가?
2. 절차적 코드 수정 체험: 절차적 행맨에서 “이미 시도한 글자” 기능을 추가해

보라. 어디에 코드를 넣어야 할지 찾기 쉬운가?

3. 새 게임 만들기: 가위바위보 게임을 (1) 절차적으로, (2) 클래스로 각각 만들어 비교해 보라.